НИУ ВШЭ НН

ИМИКН

EcoFFT

Архитектура, API и руководство разработчика

Прямое и обратное быстрое преобразование Фурье

Eco ACOM · Linux x86_64 · версия 1.0.0

Автор: Зиновьев Дмитрий Константинович

Группа: 24ТИДИ

Проверяющий: Башев Владимир Николаевич

Нижний Новгород, 2026

Содержание

1. Назначение и возможности

2. Математическая модель

3. Алгоритмы DIT и DIF

4. Произвольная длина: алгоритм Блюстейна

5. Тригонометрия и численные типы без libc

6. Архитектура ACOM и поддержка Eco

7. Интерфейсы и контракты

8. Сервисный и объектный подходы

9. Память: модель и измерения

10. Организация проверок

11. Примеры и сравнение точности

12. Измерение производительности

13. Сборка и практическое использование

14. Устройство проекта и применение Eco.AI

15. Системный промпт модели

16. Ограничения и направления развития

17. Технические источники

1. Назначение и возможности

EcoFFT — библиотека компонентов Eco ACOM для прямого и обратного быстрого преобразования Фурье. Она предназначена для спектрального анализа комплексных сигналов, восстановления сигнала по спектру и блочной фильтрации. Приложение получает вычислительный интерфейс через штатную шину Eco; консольный клиент позволяет обрабатывать собственные данные без написания программы.

Реализованы два взаимозаменяемых компонента с единым контрактом: DIT и DIF. Для длин, не равных степени двойки, каждый использует преобразование Блюстейна. Поддерживаются N от 1 до 65536. Это ограничение делает потребление памяти предсказуемым и позволяет проверять арифметику индексов до выделения буферов.

Вычисления доступны в двух архитектурных формах. Сервис IEcoDSP получает входные и выходные массивы при каждом вызове и не сохраняет сигнал. Объект IEcoFFTPlan владеет копией сигнала и подготовленными таблицами; его можно многократно использовать для одного размера. IEcoDigitalFilter сохраняет спектр ядра циклической свертки. IEcoVector и IEcoMatrix предоставляют собственные комплексные контейнеры.

Возможность	**Публичное средство**
FFT и IFFT для комплексных данных	Семейства fft/ifft, fftf/ifftf, fftl/ifftl
Целочисленный действительный вход	ffti/iffti и fftlong/ifftlong с комплексным результатом
Повторное преобразование одного размера	IEcoFFTPlan: Load, FFT, IFFT, Store
Циклическая свертка	Сервис Filter и объект IEcoDigitalFilter
Векторы и матрицы	CreateVector, CreateMatrix, проверяемые Get/Set
Выбор реализации	Отдельные CID, статические и динамические библиотеки DIT/DIF
Пользовательские файлы	CSV-клиент eco_fft, стандартные потоки, сервисный и объектный режимы
Встраивание в приложение	Публичный C-интерфейс и собираемый пример examples/use_component.c

Целевая платформа — Linux x86_64. В Windows сборка и выполнение осуществляются внутри Docker с Linux-контейнерами. Ядро написано на C11, использует собственную комплексную арифметику и выделяет память только через IEcoMemoryAllocator1. Готовые FFT-библиотеки, стандартные заголовки и функции libc/libm в вычислительный компонент не входят.

Матрица предоставляет построчное хранение не более 65536 элементов. Двумерное FFT и потоковая обработка с историей задержек не входят в текущий API. Последующие разделы фиксируют математические соглашения, управление состоянием, численные пределы и правила интеграции.

2. Математическая модель

Комплексное число представлено собственной структурой с действительной и мнимой частями. Для двух чисел a и b произведение вычисляется как (a.re·b.re − a.im·b.im, a.re·b.im + a.im·b.re). Заголовок complex.h и готовые комплексные операции не используются.

Прямое преобразование определено для k=0,…,N−1:

$X[k] = \Sigma(n=0\ldots N-1)\ x[n] \cdot \exp(-i\cdot 2\pi\cdot k\cdot n/N)$.

Обратное преобразование определено для n=0,…,N−1:

$x[n] = (1/N) \cdot \Sigma(k=0\ldots N-1)\ X[k] \cdot \exp(+i\cdot 2\pi\cdot k\cdot n/N)$.

Отрицательный знак соответствует FFT, положительный — IFFT. Нормировка 1/N выполняется только в обратном преобразовании. Результат расположен в обычном порядке частот: нулевая частота в элементе 0; fftshift не применяется. Контракт обеспечивает IFFT(FFT(x)) ≈ x с учетом округления. Выбранный коэффициент нормировки зафиксирован в интерфейсе, тестах и эталоне: соглашения разных библиотек могут отличаться [5].

Для импульса x[0]=1 остальные элементы равны нулю, и спектр состоит из единиц. Для постоянного сигнала x[n]=c ненулевым является только X[0]=Nc. Комплексная гармоника exp(i·2π·r·n/N) должна дать пик в позиции r. Эти свойства проверяют знак, порядок частот и коэффициент нормировки независимо от обратного преобразования. Только тест roundtrip был бы недостаточен: одинаковые ошибки FFT и IFFT могли бы взаимно компенсироваться.

Для действительного сигнала спектр в общем случае комплексный. Поэтому целочисленные варианты принимают int или long, а возвращают EcoComplexD. Округлять спектр до целых чисел нельзя без потери результата. В iffti и ifftlong целые входы трактуются как действительные значения спектра; восстановленный сигнал также комплексный.

3. Алгоритмы DIT и DIF

Основа radix-2 — разложение преобразования размера N на два преобразования N/2 и стадию объединения. Работа Cooley–Tukey служит исходным математическим ориентиром [3]. Реализация выполнена самостоятельно, готовый код FFT из SDK или внешней библиотеки не включается.

3.1. Decimation in Time

В DIT сначала выполняется битовая перестановка индексов. Затем длина блока последовательно увеличивается: 2, 4, 8, …, N. Для каждой пары берутся u=a[k] и t=W·a[k+L/2], после чего записываются u+t и u−t. Здесь W=exp(−i·2π·j/L). Для IFFT знак мнимой части W меняется на противоположный.

```
bit_reverse(data)
for L = 2, 4, ..., N:
   for each block of length L:
      for j = 0, ..., L/2 - 1:
         u = data[block+j]
         t = W[j*N/L] * data[block+j+L/2]
         data[block+j] = u+t
         data[block+j+L/2] = u-t
if inverse: divide every element by N
```

Перестановка до вычислений обеспечивает естественный порядок результата. Ядро итеративное и не использует рекурсивный стек. Поворотные множители вычисляются при создании численного плана и затем только читаются.

3.2. Decimation in Frequency

В DIF длина блока уменьшается: N, N/2, …, 2. Сначала вычисляются сумма u+v и разность u−v, затем разность умножается на W. Выход промежуточных стадий находится в битово переставленном порядке, поэтому финальная перестановка выполняется после всех бабочек.

```
for L = N, N/2, ..., 2:
   for each block of length L:
      for j = 0, ..., L/2 - 1:
         u = data[block+j]
         v = data[block+j+L/2]
         data[block+j] = u+v
         data[block+j+L/2] = (u-v) * W[j*N/L]
bit_reverse(data)
if inverse: divide every element by N
```

DIT и DIF реализованы двумя разными циклами бабочек. Это не два названия одного вызова. Каждый алгоритм опубликован отдельной фабрикой и отдельной библиотекой с собственным CID. Код жизненного цикла, контейнеров и универсальных преобразований типов общий; это устраняет дублирование архитектурной обвязки, сохраняя выбор алгоритма в клиенте.

3.3. Асимптотика

Для $N=2^p$ число стадий равно $\log_2 N$. На каждой стадии выполняется N/2 комплексных бабочек. Следовательно, время преобразования O(N log N), битовой перестановки O(N), подготовки таблиц O(N). Внутренний буфер и таблицы занимают O(N) памяти; рекурсивная глубина отсутствует. Прямой ДПФ требует $O(N^2)$ комплексных операций и используется только в независимом тестовом эталоне.

4. Произвольная длина: алгоритм Блюстейна

Простой radix-2 не определяет вычисление при N=7 или N=127. Вместо скрытого дополнения исходного сигнала нулями, которое изменило бы частотную сетку, реализовано преобразование Блюстейна [4]. Оно вычисляет именно N-точечное ДПФ, используя вспомогательную свертку большей длины.

Из тождества $2kn=k^2+n^2-(k-n)^2$ следует:

$c[n] = \exp(-i\cdot\pi\cdot n^2/N)$, $a[n]=x[n]c[n]$, $b[t]=\exp(+i\cdot\pi\cdot t^2/N)$.

$X[k] = c[k] \cdot \Sigma(n=0\ldots N-1)\ a[n] \cdot b[k-n]$.

Выбирается M — минимальная степень двойки не меньше 2N−1. Положительные индексы b записываются в начало массива, отрицательные — в позиции M−t. Остальные элементы нулевые. Свертка получается как IFFT_M(FFT_M(a)·FFT_M(b)); первые N результатов умножаются на c[k]. Внутренняя IFFT делится на M, а внешняя обратная операция использует сопряжение и отдельный делитель N. Смешивать эти две нормировки нельзя.

В реализации сначала вычисляется целочисленный остаток $n^2 \bmod 2N$, затем угол. При $N \le 65536$ квадрат максимального индекса помещается в uint32_t. Угол не растет пропорционально N^2, что уменьшает ошибку приведения диапазона. В плане сохраняются chirp, FFT(b), таблица поворотных множителей и рабочий буфер. Повторное применение подготовленного численного плана не выделяет память.

Поскольку M<4N, сохраняются O(N log N) времени и O(N) памяти. Однако константы выше, чем для прямого radix-2. Поэтому соседние длины 65535 и 65536 могут иметь существенно разное время и объем памяти. В обоих компонентах используется одна схема Блюстейна, но ее внутренние FFT выполняются выбранным DIT либо DIF.

5. Тригонометрия и численные типы без libc

В компоненте отсутствуют sin, cos, pow, sqrt, malloc, memcpy и другие функции стандартной библиотеки. Численное ядро получает ограниченный угол, приводит его по четвертям к интервалу $[-\pi/4, \pi/4]$, отдельно вычисляет ряды синуса и косинуса и восстанавливает знаки и перестановку функций.

Начальные члены $s_0=x$ и $c_0=1$. Для $j=1,\ldots,12$ используются рекуррентные формулы членов ряда:

$s_j = s_{j-1} \cdot (-x^2)/((2j)(2j+1))$; $c_j = c_{j-1} \cdot (-x^2)/((2j-1)(2j))$.

Суммируются начальные значения и 12 обновлений. В этой области остаточный член имеет порядок меньше $3 \cdot 10^{-29}$; фактическая точность ограничена округлением long double, а не только остатком ряда. Каждый поворотный множитель рассчитывается независимо. Длинная цепочка умножений W←W·Wstep, накапливающая ошибку фазы, не применяется.

Внутренняя арифметика всех вариантов использует long double. На проверенной платформе это 64 значащих двоичных разряда, 16 байт хранения скаляра и 32 байта EcoComplexL. float и double расширяются перед расчетом, затем результат сужается один раз. Это архитектурное решение повышает точность промежуточных операций, но означает, что float-вариант не является оптимизированным вычислением в одинарной точности. Его скорость нельзя напрямую сопоставлять с SIMD FFT для float.

Для float ограничен модуль каждой входной части величиной 10^{30}; для остальных типов — 10^{100}. NaN и бесконечности отвергаются упорядоченными сравнениями. Векторы и внутреннее состояние плана также ограничены величиной 10^{100}. План проверяет результат

до замены старого сигнала. Очень большой допустимый вход сервисного FFT может дать результат за пределом входного диапазона следующего IFFT; это явно ограничивает область обещания roundtrip. Для произвольных амплитуд преобразование масштабировать автоматически не пытается.

6. Архитектура ACOM и поддержка Eco

Приложение использует настоящий Eco.System1. Функция main принадлежит его официальной статической библиотеке; точка входа консольного клиента имеет сигнатуру int16_t EcoMain(IEcoUnknown*). Через QueryInterface получается IEcoSystem1, затем IEcoInterfaceBus1. На шине регистрируется фабрика менеджера памяти и фабрика выбранного математического компонента. QueryComponent по CID возвращает IEcoDSP.

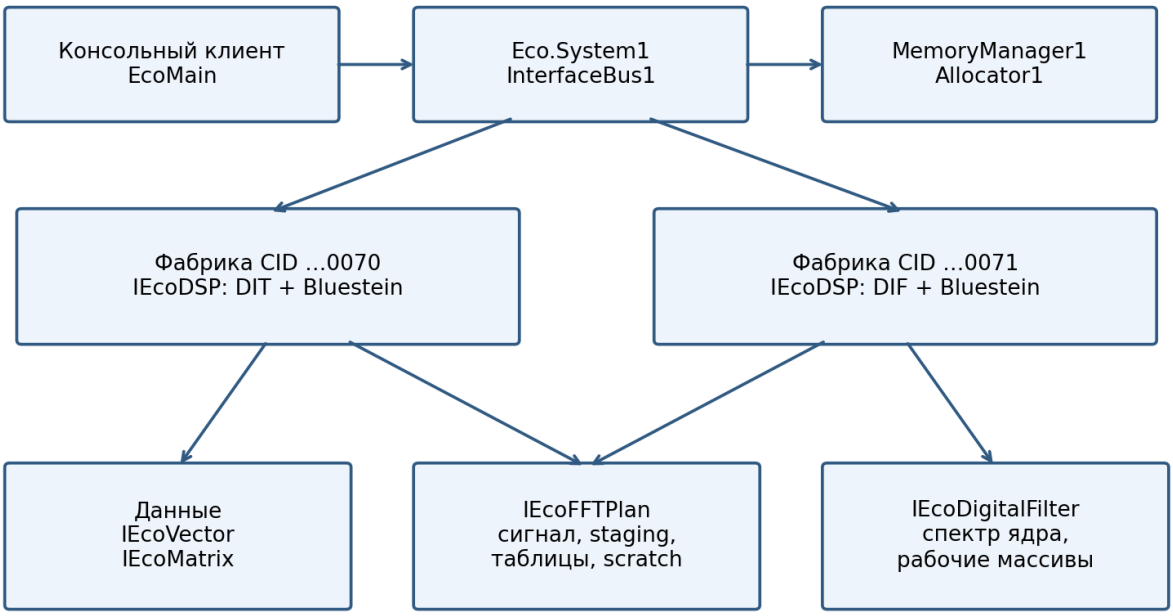


Рисунок 1 — загрузка через штатный Eco runtime и владение объектами.

Первые три метода каждой таблицы виртуальных функций — QueryInterface, AddRef и Release. Успешное создание или QueryInterface возвращает одну принадлежащую вызывающему ссылку. Release уменьшает счетчик; при нуле освобождаются дочерние буферы, объект и удерживаемая ссылка на аллокатор. Интерфейс размещен первым полем объекта, поэтому указатель интерфейса преобразуется в указатель владельца без смещения. Все объекты имеют каноническую идентичность IEcoUnknown.

Неподдерживаемый IID возвращает ошибку и обнуляет выходной указатель. Фабрика явно отвергает ненулевой outer: COM-агрегация не поддерживается, поэтому запрос отклоняется. Методы Add и Subtract предоставляют сложение и вычитание вещественных значений.

Сущность	Идентификатор
CID DIT	C345FB78-7071-4410-9821-A6EF25310070
CID DIF	C345FB78-7071-4410-9821-A6EF25310071
IID IEcoDSP	C345FB78-7071-4410-9821-A6EF25310001
IID IEcoVector	C345FB78-7071-4410-9821-A6EF25310002
IID IEcoMatrix	C345FB78-7071-4410-9821-A6EF25310003
IID IEcoFFTPlan	C345FB78-7071-4410-9821-A6EF25310004
IID IEcoDigitalFilter	C345FB78-7071-4410-9821-A6EF25310005

UGUID имеет преамбулу 0x01, длину данных 0x10 и 16 байт идентификатора; sizeof(UGUID)=18. Для динамической загрузки экспортируется официальный GetIEcoComponentFactoryPtr. В статических библиотеках присутствуют имена фабрик с полным CID. Клиент поддерживает оба способа; при выгрузке сначала отменяет регистрацию на шине, а затем освобождает отображение библиотеки.

Оригинальный SDK depend.h в ветви Linux подключает стандартные заголовки. Чтобы соблюсти строгую границу даже на этапе препроцессора, авторские единицы трансляции собираются с адаптером eco_freestanding.h. Он задает примитивные ABI-типы Linux x86_64 и отключает только платформенный depend.h. Официальные определения UGUID, фабрики, шины и аллокатора остаются исходными. Клиент компилируется с оригинальным SDK; размеры типов проверяются в тестах.

Компонент написан на C11 и использует статические проверки ABI и inline-функции. Официальные структуры SDK сохранены. Стандартные библиотеки применяются в консольном клиенте, операциях ввода/вывода и независимом эталоне math.h; авторские .so не имеют этих зависимостей.

7. Интерфейсы и контракты

Полный публичный контракт находится в include/IEcoDSP.h. Во всех методах первый параметр me обозначает экземпляр интерфейса. В таблице input — входной массив, output — буфер результата длины N, out — адрес указателя на создаваемый объект. Внешними массивами владеет клиент; компонент не освобождает их и не сохраняет после сервисного вызова.

Методы	Вход → выход	Назначение
fft / ifft	EcoComplexD[N] → EcoComplexD[N]	Двойная точность интерфейса
fftf / ifftf	EcoComplexF[N] → EcoComplexF[N]	Одинарная точность интерфейса
fftl / ifftl	EcoComplexL[N] → EcoComplexL[N]	Расширенная точность
ffti / iffti	int[N] → EcoComplexD[N]	Действительные целочисленные данные

Методы	Вход → выход	Назначение
fftlong / ifftlong	long[N] → EcoComplexD[N]	Целые типа long
CreateVector	N → IEcoVector	Собственный комплексный вектор
CreateMatrix	rows, cols → IEcoMatrix	Плотная матрица в построчном порядке
CreatePlan	N → IEcoFFTPlan	Объект с таблицами и внутренним сигналом
CreateFilter	IEcoVector → IEcoDigitalFilter	Снимок ядра циклической свертки
Filter	signal, kernel, output	Сервисная свертка без сохранения ядра
Add / Subtract	long double, long double → long double	Базовые арифметические операции
Algorithm / MemoryBytes	экземпляр → uint32_t	Выбранный алгоритм / объем объекта

Пример вызова метода в стиле ACOM:

```
EcoComplexD input[4] = {{1,0},{2,0},{3,0},{4,0}};
EcoComplexD spectrum[4];
int16_t rc = dsp->pVTbl->fft(dsp, 4, input, spectrum);
/* rc == 0: spectrum = {10, -2+2i, -2, -2-2i}. */
```

Точное совпадение input и output поддерживается. Частичное перекрытие не является обещанным контрактом публичного интерфейса, даже если внутренний scratch делает некоторые такие случаи работоспособными. При ошибке аргумента или выделения памяти сервис не записывает результат. У произвольного внешнего IEcoVector метод Set может завершиться ошибкой после нескольких успешных записей: такую ошибку Store/Apply передает клиенту; полного отката стороннего контейнера контракт не обещает.

Код	Значение
0, ECO_OK	Успех
−1, ECO_E_ARGUMENT	Нулевой или некорректный аргумент
−2, ECO_E_MEMORY	Отказ выделения памяти
−3, ECO_E_RANGE	Длина, индекс, размерность или значение за пределами контракта
−4, ECO_E_STATE	План не загружен либо внутренний план занят
−5, ECO_E_INTERFACE	Интерфейс не поддерживается

Интерфейсы данных имеют Get/Set с проверкой индексов и MemoryBytes. Нулевые размеры запрещены. Произведение rows·cols проверяется делением до умножения, что исключает переполнение. Объекты однопоточные: совместное использование одного плана или счетчика ссылок требует внешней синхронизации; независимые экземпляры имеют собственные рабочие буферы.

8. Сервисный и объектный подходы

8.1. Сервис без состояния

IEcoDSP хранит только таблицу методов, счетчик ссылок, выбранный алгоритм и ссылку на аллокатор. Сигнал, преобразованные данные и коэффициенты между вызовами не сохраняются. Каждый fft/ifft создает численный план и буфер преобразования типов, рассчитывает результат и освобождает временную память. Сервис является stateless относительно вычислительных данных; наличие инфраструктурного счетчика ссылок не делает его состоянием сигнала.

Преимущество сервиса — короткий вызов и независимость от порядка предшествующих вычислений. Он удобен для единичного преобразования или постоянно меняющегося N. Цена — повторное выделение памяти, вычисление таблиц и, для Блюстейна, подготовка FFT ядра свертки при каждом вызове.

8.2. Объект IEcoFFTPlan

CreatePlan фиксирует N и выбранный алгоритм. Объект содержит numerical kernel, собственный signal[N], staging[N] и признак loaded. Load читает внешний вектор в staging, проверяет все значения и только после успеха заменяет внутренний сигнал. Изменение исходного вектора после Load не изменяет объект.

```
/* vec and result were created via CreateVector, plan via CreatePlan. */
rc = plan->pVTbl->Load(plan, vec);
if (rc == 0) rc = plan->pVTbl->FFT(plan);
if (rc == 0) rc = plan->pVTbl->Store(plan, result);
if (rc == 0) rc = plan->pVTbl->IFFT(plan);
/* Release each successfully created interface exactly once. */
```

FFT/IFFT не принимают массив снаружи: они преобразуют сохраненные данные объекта. Store копирует текущее состояние в предоставленный вектор. До Load методы преобразования и Store возвращают ECO_E_STATE. Ошибка Load или преобразования не заменяет предыдущее корректное состояние. Два плана, созданные одним сервисом, не разделяют сигналы и рабочие массивы.

План полезен при многократной работе с одним размером: подготовленные коэффициенты сохраняются, вызовы не выделяют новую память. Но клиент должен управлять последовательностью состояний и временем жизни объекта. Дополнительные Get/Set и копирования могут уменьшить выигрыш от кэша; это отдельно измерено.

8.3. Объект IEcoDigitalFilter

При CreateFilter вектор коэффициентов копируется и преобразуется в спектр H. Apply(signal,result) вычисляет Y=FFT(signal)·H и затем IFFT(Y). Повторный Apply не читает исходное ядро и не пересчитывает его спектр. Изменение коэффициентов в исходном векторе после создания не меняет объект. Сервисный Filter создает аналогичный временный объект на время одного вызова.

Операция представляет N-точечную циклическую свертку: $y[n]=\Sigma\, h[j]\cdot x[(n-j) \bmod N]$. Пример $h=\{0.5,0.5,0,\ldots\}$ дает $y[n]=(x[n]+x[(n-1) \bmod N])/2$. Это блочный FIR-пример с

неизменяемыми коэффициентами; он не моделирует потоковый IIR, историю задержек или линейную свертку без циклического перехода.

Перед обратным преобразованием проверяется диапазон спектрального произведения, а перед записью наружу — весь результат. Неверный вход не разрушает кэш. Демонстрация и UnitTest проверяют повторные вызовы, одинаковые результаты SOA/OOA, совпадение входного и выходного векторов и неизменность сохраненных коэффициентов.

9. Память: модель и измерения

MemoryBytes сообщает сумму запрошенных байт объекта и всех принадлежащих ему буферов. Служебные заголовки аллокатора, память кода, системной шины и клиентские массивы в эту величину не входят. Корректность метода проверяется независимым счетчиком запросов, который делегирует реальное выделение официальному Eco-аллокатору.

На проверенном ABI sizeof(EcoComplexL)=32. Сервис занимает 32 байта, вектор — 32+32N, матрица — 40+32·rows·cols. Численное ядро K(N) при $N=2^p \geq 2$ занимает 64+48N байт: служебные поля, N/2 коэффициентов и N элементов scratch. План добавляет 56+64N байт на объект и два массива. Итого план: 120+112N. Для N=1 размер таблицы имеет отдельный минимум в один элемент.

Для Блюстейна K(N)=64+80M+32N, где M=nextpow2(2N−1). План занимает 120+80M+96N. Фильтр отличается от плана размером служебной структуры: 48 вместо 56 байт, при тех же двух собственных N-массивах и численном ядре.

N	Сервис, байт	Вектор, байт	План, байт	Фильтр, байт
16	32	544	1912	1904
127	32	4096	32792	32784
1024	32	32800	114808	114800
65535	32	2097152	16777240	16777232
65536	32	2097184	7340152	7340144

У stateless-сервиса малая постоянная память, но временная память не равна нулю. Пик на один расчет равен 32+32N+K(N) без внешних входного и выходного массивов. План сохраняет все свои массивы между вызовами. Если сравнивать полный клиентский сценарий, дополнительно учитываются два внешних массива у SOA или два объекта-вектора у OOA; сравнивать только 32 байта сервиса с полным планом как одинаковые пики было бы неверно.

10. Организация проверок

Тестовый клиент получает оба компонента через настоящую Eco InterfaceBus. Проверки запускаются для статической линковки и для загрузки двух .so через dlopen с последующей регистрацией фабрик на шине. Дополнительно тот же клиент компилируется

с AddressSanitizer и UndefinedBehaviorSanitizer; инструментируется авторский код и клиент, но не закрытые предварительно собранные библиотеки SDK.

Запуск	Проверок	Ошибок	Итог	Журнал
Статическая сборка	9029	0	PASS	build/results/tests.log
Динамическая загрузка	9029	0	PASS	build/results/dynamic-tests.log
ASan + UBSan	9029	0	PASS	build/results/sanitizer.log

Это 9029 проверок в каждом запуске, а не 27087 различных тестовых сценариев. Внутри запуска 5093 проверки относятся к отказам, памяти и времени жизни объектов; tracking allocator обработал 722 вызова выделения с инъекцией отказов. Оставшиеся проверки относятся к математике, данным и публичному API.

10.1. Независимые математические проверки

Эталон точности — прямое ДПФ $O(N^2)$, написанное в клиенте с sinl/cosl из math.h. В нем нет вызовов проверяемого ядра. Фаза вычисляется непосредственно по паре индексов k,n; для уменьшения погрешности аргумент приводится целочисленным остатком modulo N. Сравниваются и прямое, и обратное преобразования, а не только их композиция.

Для каждого алгоритма берутся N={1,2,3,5,7,8,16,31,64,127,256,257}. Для каждого размера используются пять сигналов: импульс, константа, действительная гармоника, комплексная гармоника и псевдослучайная комплексная последовательность с фиксированным начальным состоянием. Проверяются пять типов и два направления: всего 600 строк измерений для DIT и 600 для DIF.

Эталон получает именно представимые входные значения выбранного типа. Например, float сначала округляется до float, затем расширяется для ДПФ; это отделяет ошибку алгоритма от исходного квантования данных. Для int используется целочисленный сигнал, для long — его увеличенная версия. Величины ошибок long поэтому нельзя интерпретировать как точность float на единичном диапазоне.

Для N=4096,65535,65536 проверяются roundtrip и равенство Парсеваля. Аналитические тесты отдельно проверяют положение пиков, знак мнимой части и нормировку. In-place запуск сравнивается с отдельным выходным массивом.

10.2. Контракт, жизненный цикл и ошибки

Проверяются нулевые указатели, N=0 и N>65536, NaN/Inf, выход за границы вектора/матрицы, неизвестные IID, выходные указатели при ошибке, состояние плана до Load, несовпадение размеров, владение ссылками и неизменность внешних данных после их копирования. В тесте внешний вектор специально возвращает ошибку Get: старое состояние плана должно остаться доступным.

Tracking allocator фиксирует каждый запрос и освобождение, суммарный объем и число ссылок. Для конструкторов сервиса, вектора, матрицы, плана и фильтра последовательно

моделируется отказ на каждой точке выделения до первого успешного полного создания. Аналогично проверяются временные выделения сервисных FFT и Filter. После отказа не должно оставаться частично созданных объектов или потерянных ссылок. Повторные операции готового плана и фильтра не должны увеличивать число Alloc.

10.3. Запрет libc

Сборка компонента использует -ffreestanding, -fno-builtin и -fno-tree-loop-distribute-patterns, чтобы компилятор не подменял написанный цикл вызовом memcpy/memset. Динамические библиотеки линкуются с -nostdlib, -nodefaultlibs и -Wl,-z,defs. Проверяются исходники, фактические зависимости препроцессора и все неопределенные символы, а не только короткий список известных функций.

Компонент	**Неопределенных символов**	**DT_NEEDED**	**Итог**
0070	0	нет	PASS
0071	0	нет	PASS

Аудит охватил 8 авторских файлов .c/.h. В зависимостях препроцессора компонента нет системных заголовков /usr/. Сами оригинальные библиотеки Eco runtime являются внешними поставляемыми зависимостями: проверка не утверждает, что их внутренняя реализация не использует ОС или libc. Полный результат и SHA256 библиотек при повторной проверке записываются в build/results/audit.json.

10.4. Внешний интерфейс и пример интеграции

CSV-клиент прошел 105 проверок в Release, включая статическую и динамическую загрузку. Дополнительно 68 проверок статического клиента выполнены под ASan/UBSan. Результаты сравниваются с независимым ДПФ на Python для DIT/DIF, сервисного и объектного API, прямого и обратного преобразований. Проверены N=1,4,7, композиция FFT/IFFT и максимальная длина 65536 на импульсном сигнале.

Отдельно проверены чистый CSV в stdout, стандартный ввод, файлы с пробелами в пути, BOM/CRLF, комментарии, пустой сигнал, неверные поля, NaN/Inf, превышение числового диапазона, слишком длинная строка, N>65536, отсутствие входа и библиотеки, ошибки открытия и записи выхода. Во всех сценариях неверного входа ранее существовавший выходной файл сохранил содержимое. Пример examples/use_component.c собран и выполнил FFT/IFFT с проверкой известного результата.

Журналы проверок CLI при повторном запуске сохраняются в build/results/cli-checks.json и cli-sanitizer.json. Числа обозначают проверки сценариев данного клиента и не складываются с числом проверок математического компонента как мера покрытия кода.

11. Примеры и сравнение точности

11.1. Сравнение с прямым ДПФ на math.h

Метрика абсолютной ошибки: $Eabs=\max_k(\max(|Re(Y_k-Yref_k)|, |Im(Y_k-Yref_k)|))$. Масштабированная ошибка делит эту величину для каждого k на $\max(1, |Re(Yref_k)|, |Im(Yref_k)|)$. Она не является чистой относительной ошибкой возле нулевого спектрального коэффициента. Roundtrip сравнивает исходный представимый сигнал с IFFT(FFT(x)).

Количество десятичных знаков d в таблице выбрано консервативно по условию $Eabs<0.5\cdot10^{-d}$. Это эмпирическая оценка для испытанного набора и его амплитуд; она не является доказательством такой же точности на всех допустимых входах. В таблице показан максимум по обоим алгоритмам, всем размерам и сигналам; FFT и IFFT разделены.

Тип	Направление	Eabs	Escaled	Знаков d	Roundtrip
float	FFT	5.612e-07	5.818e-08	5	2.553e-08
float	IFFT	1.855e-08	1.855e-08	7	—
double	FFT	2.248e-15	1.084e-16	14	4.982e-17
double	IFFT	3.415e-17	3.415e-17	16	—
long double	FFT	5.551e-17	3.781e-17	15	1.735e-18
long double	IFFT	3.271e-19	3.271e-19	18	—
int	FFT	7.532e-13	4.072e-15	11	6.863e-15
int	IFFT	3.431e-15	1.022e-16	14	—
long	FFT	4.329e-06	3.293e-08	5	5.402e-08
long	IFFT	2.712e-08	3.020e-10	7	—

Для long double максимальная абсолютная ошибка прямого преобразования равна 5.551e-17, обратного — 3.271e-19. Для double ошибка прямого преобразования составляет 2.248e-15; для float — 5.612e-07. Различие объясняется прежде всего форматом выдаваемого результата. Даже расширенный эталон имеет конечную точность; таблица измеряет расхождение двух вычислительных реализаций, а не ошибку относительно бесконечно точной арифметики. Проверки больших N отдельно подтверждают восстановление и энергетическое соотношение; их нельзя смешивать с приведенной таблицей прямого сравнения с ДПФ.

Следующий пример использует x={1,2,3,4,5,6,7,8}. В нем X[0]=36, X[4]=−4, остальные коэффициенты образуют комплексно-сопряженные пары. Для каждой строки сравнение с math.h ДПФ сохранено в build/results/demo-dit.csv; аналогичная таблица имеется для DIF.

k	x[k]	Re FFT	Im FFT	Re IFFT	Re эталона
0	1	36.000000000	0.000000000	1.000000000	36.000000000
1	2	-4.000000000	9.656854249	2.000000000	-4.000000000
2	3	-4.000000000	4.000000000	3.000000000	-4.000000000
3	4	-4.000000000	1.656854249	4.000000000	-4.000000000
4	5	-4.000000000	0.000000000	5.000000000	-4.000000000

k	x[k]	Re FFT	Im FFT	Re IFFT	Re эталона
5	6	-4.000000000	-1.656854249	6.000000000	-4.000000000
6	7	-4.000000000	-4.000000000	7.000000000	-4.000000000
7	8	-4.000000000	-9.656854249	8.000000000	-4.000000000

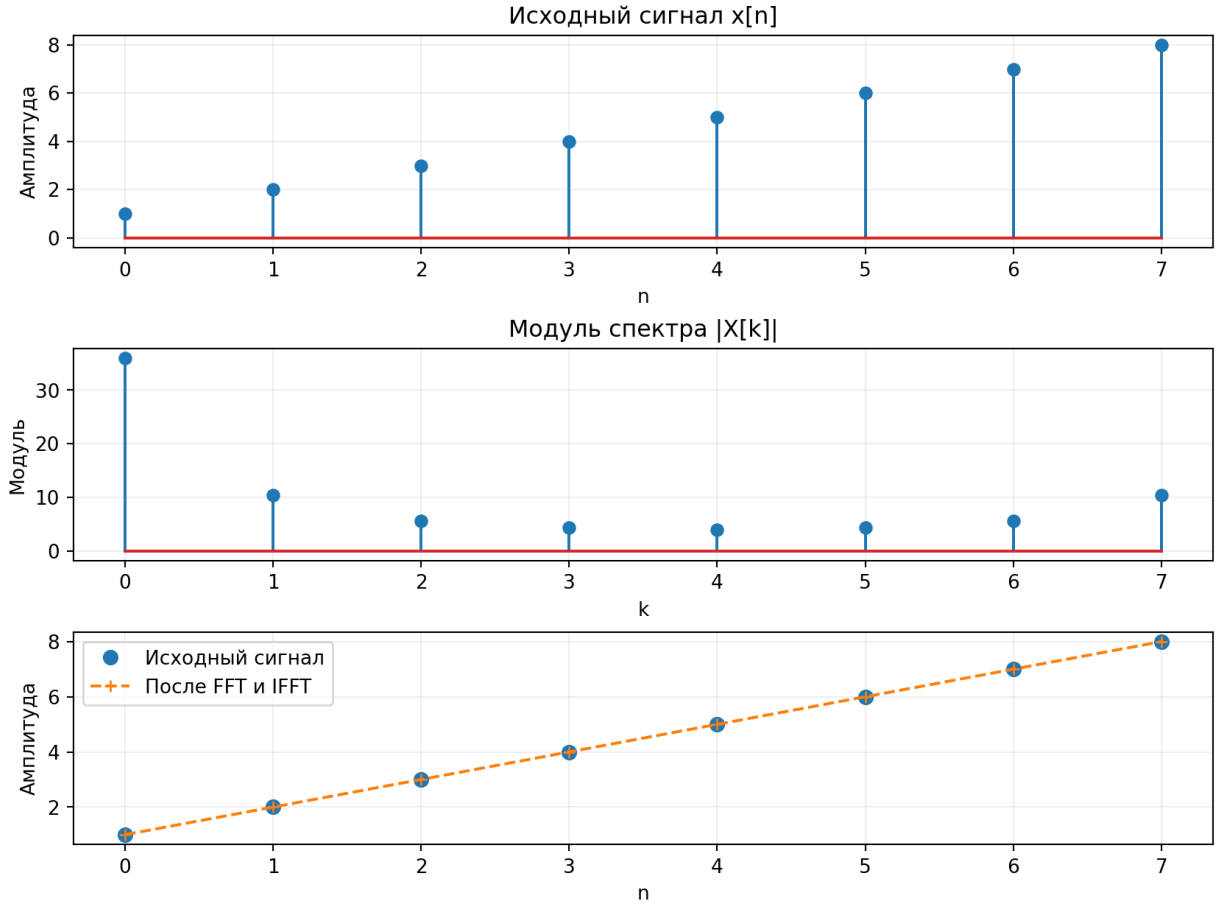


Рисунок 2 — исходные отсчеты, модуль FFT и восстановление IFFT для примера N=8.

В консольной демонстрации все коэффициенты выведены с 12 знаками после запятой. Кроме сравнения с ДПФ клиент проверяет точное совпадение результатов SOA и OOA для этого примера. Восстановленные действительные значения совпадают с исходными до показанных 9 знаков; исходные CSV сохраняют до 18 значащих цифр.

11.2. Сравнение с FFTW

Дополнительный независимый ориентир — библиотека FFTW, фактическая версия fftw-3.3.10. Клиент client/compare_fftw.c вызывает fftwl_plan_dft_1d и fftwl_execute через публичный API [5–7]. Обе стороны получают одинаковые комплексные значения long double. FFTW_BACKWARD возвращает ненормированный результат; клиент делит обе его части на N. Сравнение выполняется непосредственно для FFT и IFFT, а не только по восстановлению сигнала.

Проверены 18 длин: 1, 2, 3, 5, 7, 8, 16, 31, 64, 127, 256, 257, 1023, 1024, 4096, 16384, 65535 и 65536. Для каждой длины используются импульс, константа, действительная косинусоида, комплексная гармоника и детерминированный псевдослучайный сигнал.

DIT/DIF, FFT/IFFT и SOA/OOA дают 720 сравнений; все завершились успешно. Порог приемки Escaled<$2\cdot10^{-14}$; в таблице приведены фактические максимумы, а не значения допуска. Те же 720 сравнений отдельно прошли под ASan/UBSan, включая путь создания и освобождения планов FFTW. Инструментированы авторский код и клиент; готовые двоичные библиотеки Eco runtime и FFTW не инструментированы.

Алгоритм	API	Операция	Eabs	Escaled	Знаков d
DIF	OOA	FFT	1.421e-14	5.432e-15	13
DIF	OOA	IFFT	2.711e-19	2.711e-19	18
DIF	SOA	FFT	1.421e-14	5.432e-15	13
DIF	SOA	IFFT	2.711e-19	2.711e-19	18
DIT	OOA	FFT	1.421e-14	5.911e-15	13
DIT	OOA	IFFT	3.116e-19	3.116e-19	18
DIT	SOA	FFT	1.421e-14	5.911e-15	13
DIT	SOA	IFFT	3.116e-19	3.116e-19	18

Количество знаков d вычислено по тому же условию Eabs<$0.5\cdot10^{-d}$ и ограничено 18 для данной платформы. Оно описывает только испытанные сигналы. Это расхождение двух реализаций конечной точности, а не доказательство ошибки относительно точного математического значения. Все 720 строк сохраняются в build/results/fftw-accuracy.csv. Пять публичных числовых типов дополнительно проверяются прямым ДПФ из предыдущего подраздела; сравнение FFTW здесь выполнено для long double.

12. Измерение производительности

12.1. Методика и среда

Для сравнения со стандартным аналогом используется FFTW [5–7]. Она подключается только к отдельному исполняемому файлу eco_fft_reference и не входит в компоненты, их публичный API или основной клиент eco_fft. Прямой ДПФ и рекурсивный FFT на sinl/cosl сохранены в обычном режиме --benchmark как дополнительные алгоритмические ориентиры. Приведенные далее таблицы получены реальным запуском FFTW и EcoFFT в одном процессе.

Параметр	Фактическое значение
UTC выполнения	2026-10-11T22:02:33.479958+00:00
CPU	AMD Ryzen 5 4600H with Radeon Graphics
Платформа	Linux-6.18.40.1-microsoft-standard-WSL2-x86_64-with-glibc2.41
Компилятор	cc (Debian 14.2.0-19) 14.2.0
CMake	cmake version 3.31.6
Оптимизация	Release (-O3 -DNDEBUG), без fast-math
Тип арифметики	long double; LDBL_MANT_DIG=64, sizeof(long double)=16
FFTW	fftw-3.3.10; FFTW_ESTIMATE, один поток

Используются CLOCK_MONOTONIC, прогрев и семь измеряемых пакетов; результат — медиана микросекунд на операцию. Для каждой метрики число повторов удваивается до длительности прогревочного пакета не менее 10 мс либо до 65536 повторов. В измеряемых пакетах сохраняется выбранное число повторов. Оно записано отдельными колонками CSV. Так короткие операции не сравниваются по одному вызову таймера. Результат потребляется вне измеряемого интервала; ошибка API, нечисловой результат или сбой записи дают ненулевой код завершения.

Полный вызов SOA включает выделение временной памяти, подготовку коэффициентов, FFT/IFFT и освобождение. Полный вызов FFTW включает выделение двух выровненных буферов, создание плана, копирование входа, выполнение, нормировку IFFT и копирование выхода, уничтожение плана и освобождение буферов. Для повторного использования EcoFFT измеряет Load+FFT/IFFT+Store с уже созданными векторами и планом; FFTW измеряет копирование входа, выполнение готового плана, нормировку IFFT и копирование результата.

FFTW работает в long double через fftwl_*, в одном потоке и с FFTW_ESTIMATE. Внешний файл wisdom не загружается; внутрипроцессный кэш планировщика прогрет, поэтому полный вызов не означает холодный старт процесса. FFTW_MEASURE и другие режимы поиска плана не исследованы. Создание/удаление планов записано отдельно в CSV: eco_setup_us включает внутренние буферы и состояние EcoFFT, fftw_setup_us — один направленный план FFTW с готовыми внешними буферами. Эти две колонки описывают разный объем объектов и не равны полному времени первого преобразования.

12.2. Результаты FFT и IFFT

FFT: медиана времени, мкс. «Полный» соответствует полному вызову; «повтор» — готовому плану.

Алгоритм	N	Eco SOA	FFTW полный	Eco OOA	FFTW повтор
DIT	16	1.215	2.338	1.273	0.408
DIT	64	6.440	5.995	6.021	2.519
DIT	127	67.830	66.698	38.336	27.289
DIT	256	29.353	15.842	25.937	12.260
DIT	1023	755.574	127.544	412.542	125.184
DIT	1024	130.928	71.859	122.718	67.248
DIT	4096	706.572	438.618	530.795	326.366
DIT	16384	3213.182	2137.829	2365.765	1321.077
DIT	65535	89350.324	16130.235	51984.991	13805.614
DIT	65536	13198.383	7986.363	12885.267	7506.389
DIF	16	1.335	2.029	1.182	0.401
DIF	64	6.351	6.363	5.341	2.470
DIF	127	66.980	67.279	38.563	28.986
DIF	256	27.470	17.645	27.074	12.047
DIF	1023	690.481	130.156	424.223	112.847

Алгоритм	N	Eco SOA	FFTW полный	Eco OOA	FFTW повтор
DIF	1024	120.544	63.660	118.708	65.691
DIF	4096	496.651	278.484	569.430	317.736
DIF	16384	2382.002	2194.117	4407.011	1406.138
DIF	65535	84349.205	15136.309	48881.711	12728.179
DIF	65536	10941.275	7166.627	13317.500	7767.409

IFFT: медиана времени, мкс. «Полный» соответствует полному вызову; «повтор» — готовому плану.

Алгоритм	N	Eco SOA	FFTW полный	Eco OOA	FFTW повтор
DIT	16	1.559	2.273	1.260	0.386
DIT	64	6.883	5.697	6.286	2.414
DIT	127	65.611	59.077	43.141	31.855
DIT	256	28.161	17.937	30.232	11.650
DIT	1023	776.119	142.086	422.824	121.443
DIT	1024	127.380	83.363	141.873	63.814
DIT	4096	648.607	401.088	549.055	302.826
DIT	16384	3494.745	2049.950	2716.407	1494.698
DIT	65535	84414.269	13148.981	49327.353	14110.830
DIT	65536	11909.417	7027.664	13488.298	7351.881
DIF	16	1.325	2.111	1.279	0.438
DIF	64	6.419	5.952	5.955	2.484
DIF	127	78.423	69.930	41.907	32.873
DIF	256	28.032	17.245	27.648	12.891
DIF	1023	667.154	128.266	423.785	116.074
DIF	1024	124.668	63.680	151.311	67.000
DIF	4096	596.941	340.212	685.734	367.657
DIF	16384	2650.091	1521.181	2446.931	1642.750
DIF	65535	82017.981	14732.587	49635.651	12516.458
DIF	65536	10628.658	7621.251	12410.852	7493.344

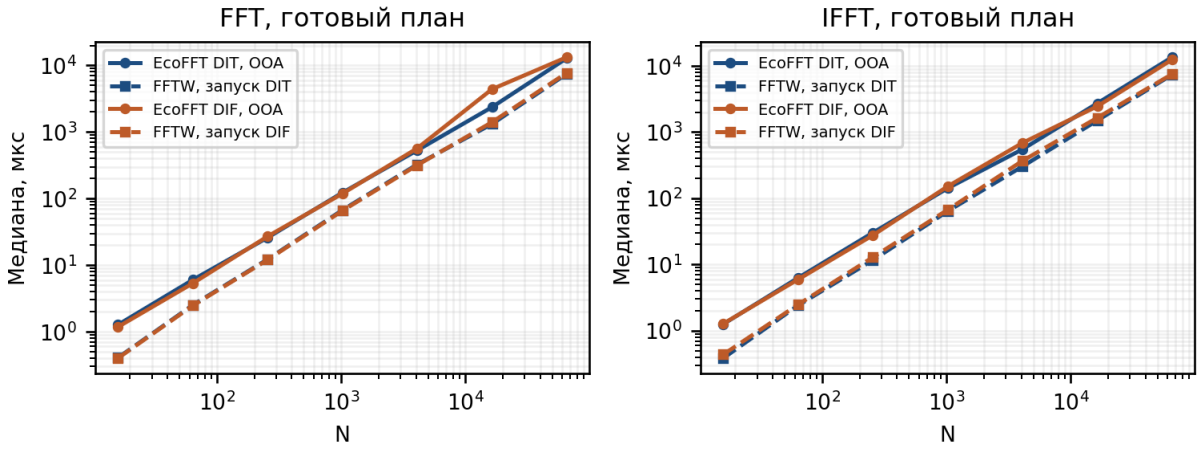


Рисунок 3 — повторные FFT/IFFT для степеней двойки; обе оси логарифмические. FFTW измерена отдельно в серии каждого компонента; нестепенные длины приведены в таблицах.

В этом запуске полный вызов FFTW быстрее SOA EcoFFT в 34 из 40 сценариев, а повторное использование FFTW быстрее OOA EcoFFT в 40 из 40 сценариев. При N=65536 для FFT DIT получены SOA 13198.383 мкс и OOA 12885.267 мкс; соответствующие времена FFTW — 7986.363 и 7506.389 мкс. При N=65535 SOA DIT занимает 89350.324 мкс: переход к свертке Блюстейна увеличивает число преобразований и размер рабочих буферов.

Сохранение плана исключает повторную подготовку коэффициентов, но Load/Store и виртуальные Get/Set создают собственные накладные расходы. FFTW применяет другую организацию вычислений и планирования, поэтому одинаковая асимптотика O(N log N) не означает одинаковое время. Эти данные характеризуют выбранные API, размеры, точность и режим планировщика на указанном CPU. Общая среда Docker Desktop, отсутствие закрепления за ядром CPU и фоновая нагрузка ограничивают точность сравнений близких времен; результаты не являются оценкой максимальной скорости FFTW или MATLAB.

Полный протокол: build/results/fftw-benchmark.csv, fftw-accuracy.csv, fftw-comparison.log и fftw-environment.json. Скрипт scripts/compare_fftw.sh воспроизводит сборку, проверку точности и измерение времени; команды приведены в разделе 13.

13. Сборка и практическое использование

13.1. Контейнерное окружение

Нужен Docker с поддержкой Linux-контейнеров; в Windows можно использовать Docker Desktop. Команды выполняются из корня проекта. Компилятор, CMake, Ninja, binutils, Python и пакет разработки FFTW устанавливаются в образ. На этапе сборки исходников контейнер работает без сети; Eco SDK уже включен в vendor/eco. Для создания образа нужен доступ к реестру базовых образов и репозиторию системных пакетов.

```
docker compose build
docker compose run --rm fft
```

Сборка создает bin/eco_fft, две динамические библиотеки bin/C345…0070.so и bin/C345…0071.so, а также две статические библиотеки lib/libC345…0070.a и lib/libC345…0071.a. Полные имена содержат CID из раздела 6. Готовые Linux-бинарники включены в поставку. Каталог build создается автоматически и содержит только промежуточные файлы и результаты пользовательских запусков.

Для сборки непосредственно в Linux x86_64 требуются GCC с C11, CMake не ниже 3.20, Ninja и binutils. Команда bash scripts/build.sh воспроизводит ту же конфигурацию. Проверочные скрипты используют Python 3.10 или новее без сторонних пакетов. Для отдельного клиента сравнения дополнительно требуются fftw3.h и libfftw3l; в Debian/Ubuntu они входят в libfftw3-dev. Обычная сборка не ищет FFTW.

13.2. Формат данных и команды

CSV содержит два числовых столбца: real и imag. Одна строка соответствует одному отсчету; размер N определяется числом строк данных. Заголовок real,imag необязателен. Используются запятая между столбцами и точка в десятичных дробях. Поддерживаются

UTF-8, BOM, LF/CRLF, пустые строки, пробелы вокруг чисел и отдельные строки комментариев с символом #.

```
real,imag
1,0
2,0
3,0
4,0
```

Комплексные части должны быть конечными и иметь модуль не выше 10^{100}. Значения, не представимые при разборе в long double, отвергаются. Предельная длина строки — 1022 байта. Пользовательский файл размещается внутри папки проекта; путь в контейнере задается относительно корня /workspace.

```
docker compose run --rm fft bin/eco_fft --fft `
  --input=examples/signal.csv --output=build/spectrum.csv
docker compose run --rm fft bin/eco_fft --ifft `
  --input=build/spectrum.csv --output=build/restored.csv `
  --algorithm=dif --architecture=object
```

Первый вызов вычисляет спектр восьми отсчетов через сервис DIT. Второй восстанавливает сигнал через объектный план DIF. Результаты доступны на компьютере в build/spectrum.csv и build/restored.csv. Выход имеет тот же формат real,imag и сохраняет до 21 значащей десятичной цифры; это формат сериализации, а не обещание 21 достоверной цифры вычисления.

Параметр	**Контракт**
--fft / --ifft	Ровно одна операция; знак и нормировка приведены в разделе 2
--input=путь	Обязательный файл либо - для стандартного ввода
--output=путь	Файл либо -; по умолчанию стандартный вывод
--algorithm=dit или dif	Выбор компонента; по умолчанию dit
--architecture=service или object	Вызов массива либо объектного плана; по умолчанию service
--dynamic=bin	Получение фабрики из .so вместо статической библиотеки
--help	Справка по всем режимам

В режиме преобразования stdout содержит только CSV. Диагностика направляется в stderr; код завершения равен нулю при успехе и ненулевой величине при ошибке. Перед открытием выходного файла вход полностью читается, проверяется и преобразуется. Поэтому недопустимый вход не создает и не перезаписывает результат. Успешный вызов заменяет указанный выходной файл; ошибка записи может оставить частичный файл. Совпадение входного и выходного пути поддерживается.

```
Get-Content examples/signal.csv | docker compose run --rm -T fft `
  bin/eco_fft --fft --input=-
```

В объектном режиме результат обязан укладываться в диапазон сохраняемого состояния плана. В сервисном режиме большой спектр может превысить разрешенный диапазон последующего входа. Перед повторным преобразованием амплитуды следует выбирать с учетом N; автоматическая нормализация прямого FFT не выполняется.

13.3. Встраивание и жизненный цикл

Файл examples/use_component.c представляет полное приложение, которому официальный Eco.System1 предоставляет main. Функция EcoMain получает системный объект, запрашивает шину интерфейсов, регистрирует менеджер памяти и фабрику DIT, получает IEcoDSP и выполняет FFT/IFFT. Пример проверяет известный спектр [10, −2+2i, −2, −2−2i] и восстановление исходного массива [1,2,3,4].

```
docker compose run --rm fft build/release/eco_fft_example
```

Приложению нужны include/IEcoDSP.h и соответствующие публичные заголовки SDK. Цель eco_fft_example в CMakeLists.txt показывает состав статической линковки. Для DIF меняются фабрика и CID; сигнатуры методов остаются прежними. Динамический вариант дополнительно открывает модуль, вызывает GetIEcoComponentFactoryPtr и регистрирует фабрику на той же шине.

Полученный интерфейс принадлежит вызывающему: каждый успешный Create* и QueryInterface требует парного Release. Созданные объекты удерживают аллокатор и могут пережить освобождение сервиса. Перед dlclose необходимо освободить все полученные из модуля объекты и отменить регистрацию фабрики, поскольку таблицы методов размещены внутри модуля. При неудаче отмены регистрации клиент оставляет модуль загруженным до завершения процесса.

13.4. Повторение проверок

```
docker compose run --rm fft bash scripts/verify.sh
docker compose run --rm fft bin/eco_fft --benchmark
docker compose run --rm fft bash scripts/compare_fftw.sh
```

Первая команда собирает проект, выполняет статические и динамические тесты, проверяет CSV-интерфейс и пример подключения, запускает ASan/UBSan и аудит зависимостей. Вторая измеряет время и память с эталонами на math.h. Третья собирает отдельный eco_fft_reference с параметром CMake -DECO_FFTW_REFERENCE=ON, выполняет 720 сравнений точности с FFTW и 40 сценариев измерения времени. Результаты записываются в build/results. Для проверки FFTW без замеров используйте build/reference/eco_fft_reference --check. После изменения Dockerfile образ пересоздается командой docker compose build.

Все журналы, сведения о среде и измерения при этих запусках создаются в build/results. Они являются воспроизводимыми результатами выполнения, поэтому не включаются в очищенную поставку. Итоговые таблицы и графики текущего запуска включены в этот документ. Реквизиты на титульном листе редактируются непосредственно в Word.

14. Устройство проекта и применение Eco.AI

Путь	Содержание
include/IEcoDSP.h	Публичные типы, идентификаторы и таблицы интерфейсов
include/eco_freestanding.h	Адаптер ABI Linux x86_64 без системных заголовков

Путь	Содержание
src/fft_kernel.c, src/fft_kernel.h	Численный план, DIT/DIF, Блюстейн и тригонометрия
src/eco_dsp.c	Сервисные методы и преобразование числовых типов
src/eco_objects.c	Векторы, матрицы, планы и фильтры
src/eco_factory.c, src/eco_internal.h	Фабрики, жизненный цикл, внутренние определения
client/eco_main.c, client/transform.c	EcoMain, параметры запуска и CSV-ввод/вывод
client/checks.c, client/faults.c	Математические проверки, ошибки, ссылки и отказы памяти
client/measure.c, client/fft_client.h	Демонстрация, измерения и объявления клиента
client/compare_fftw.c	Отдельный клиент точности и производительности с FFTW
examples/	Сигналы CSV и полноценный пример подключения
scripts/build.sh, scripts/verify.sh	Сборка и последовательность проверок
scripts/check_cli.py	Проверки CLI с независимым ДПФ на Python
scripts/compare_fftw.sh	Сборка и воспроизведение сравнения со стандартным аналогом
scripts/audit_component.py	Анализ заголовков, импортов и ELF-зависимостей
scripts/record_environment.py	Фиксация компилятора, CPU и параметров измерений
vendor/eco/	Заголовки и статические библиотеки используемых компонентов Eco
bin/, lib/	Исполняемая программа, динамические и статические компоненты
docs/EcoFFT.docx	Единый документ с архитектурой и руководством

При разработке использован официальный Eco.AI Agentic Coding Meta-Harness 0.1.3 [1,2]. Из его релизного marketplace_index.zip получены Eco.Core1, Eco.System1, Eco.InterfaceBus1, Eco.MemoryManager1 и Eco.FileSystemManagement1. Файл vendor/eco/provenance.json фиксирует URL релиза, SHA256 исходного архива и каждого сохраненного файла. Уведомления правообладателей в заголовках сохранены.

Инструментальные обертки make_eco_wizard_tool и make_eco_wizard_validate_tool применялись для создания и проверки каркаса COM/C. Проверка каркаса завершилась без ошибок. Отдельный запуск официального runtime подтвердил выделение, запись, чтение и освобождение памяти. Эти проверки относятся к совместимости с Eco; математическая корректность подтверждается тестами разделов 10–12. Реализация алгоритмов и доработка проекта выполнялись с ИИ-ассистентом; отдельная удаленная LLM-сессия внутри Harness не запускалась.

Eco.AI относится к инструментам разработки и не является зависимостью готового приложения. Для сборки и выполнения не нужны агентный сервер, его рабочая среда, ключ модели или доступ к LLM. В поставке сохранен только используемый SDK; каркасы,

исследовательские загрузки, кэши, старые архивы и промежуточные документы исключены.

Dockerfile фиксирует digest базового образа. Версии фактически установленных пакетов и компилятора записываются при проверке; пакеты системного репозитория не закреплены по отдельным версиям. Повторный запуск обязан пройти функциональные проверки, но времена измерений зависят от процессора и текущей нагрузки. Побитовое совпадение длительностей и двоичных файлов разных инструментальных окружений не является контрактом.

15. Системный промпт модели

15.1. Назначение инструкции и приемка результата

Ниже приведена системная инструкция для воспроизведения или последовательной доработки EcoFFT. Она задает целевую платформу, публичный контракт, алгоритмы, архитектуру, правила работы с памятью, состав поставки и проверяемые критерии завершения. Ее следует передавать целиком вместе с доступом к рабочей папке и официальному Eco SDK.

Промпт является спецификацией результата. Сам текст не гарантирует безошибочную генерацию вероятностной моделью: основанием приемки служат успешная сборка, независимые численные проверки, проверки памяти и аудит готовых библиотек. Инструкция требует исправлять найденные нарушения до завершения работы. Этот раздел описывает инструкцию для воспроизведения компонента, а не протокол ранее выполненной автономной LLM-сессии.

15.2. Полный текст системной инструкции

(начало системного промпта)

Ты — инженер системного программного обеспечения. Разработай и доведи до проверяемого рабочего состояния проект EcoFFT на C11: компоненты прямого и обратного быстрого преобразования Фурье для Eco ACOM, консольное приложение и документацию. Работай в корне предоставленного проекта. Если проект уже содержит реализацию, сначала прочитай публичные интерфейсы, CMakeLists.txt и код, затем сохрани бинарный контракт и внеси необходимые изменения. Если файлов нет, создай их по правилам ниже.

1. Платформа и инструменты. Целевая платформа — Linux x86_64 с ABI System V. Выполняй сборку и проверки в Docker; не устанавливай компилятор, Python, Eco.AI или глобальные утилиты в основную систему. Используй CMake не ниже 3.20, Ninja и GCC. Основной контейнер после создания образа работает без сети. Все исходники, зависимости, примеры и результаты размещай внутри проекта.

2. Eco.AI и официальный SDK. Для получения архитектурного каркаса используй Eco.AI Agentic Coding Meta-Harness из https://github.com/peerf-eco/eco-harness-releases, фиксированный релиз v0.1.3. Применяй официальные инструменты Eco Wizard и проверки каркаса. Заголовки и Linux x86_64 библиотеки бери из marketplace_index.zip этого релиза; SHA256 архива — cbe72560272d20da725015d2dbe0a3fa5fa632d18256121f9c1c928d6c34ceb6. Сохраняй происхождение и хэши используемых файлов в vendor/eco/provenance.json. Не включай стороннюю реализацию FFT. Инструменты разработки и API-ключи модели не должны требоваться для сборки готового проекта. Фиксируй только фактически выполненные вызовы инструментов и реальные проверки.

3. Компонентная модель. Используй официальные Eco.System1, Eco.InterfaceBus1, Eco.MemoryManager1 и Eco.FileSystemManagement1. Точку main предоставляет System1;

приложение определяет int16_t EcoMain(IEcoUnknown*). Получай IEcoSystem1 и шину через QueryInterface, регистрируй фабрики через RegisterComponent, создавай сервис через QueryComponent. Не заменяй runtime собственной имитацией COM. Соблюдай UGUID из SDK: преамбула 0x01, длина 0x10, 16 байт идентификатора; sizeof(UGUID)=18.

4. Фиксированные идентификаторы. Определи CID DIT как C345FB78-7071-4410-9821-A6EF25310070 и CID DIF как C345FB78-7071-4410-9821-A6EF25310071. Используй IIDs C345FB78-7071-4410-9821-A6EF25310001 для IEcoDSP, C345FB78-7071-4410-9821-A6EF25310002 для IEcoVector, C345FB78-7071-4410-9821-A6EF25310003 для IEcoMatrix, C345FB78-7071-4410-9821-A6EF25310004 для IEcoFFTPlan и C345FB78-7071-4410-9821-A6EF25310005 для IEcoDigitalFilter. Эти значения, порядок виртуальных методов и соглашения о вызове не меняй.

5. Базовый ABI. Каждый публичный интерфейс представляет структуру с единственным первым полем pVTbl — указателем на соответствующую таблицу функций. Используй ECOCALLMETHOD из SDK. Первые три метода каждой таблицы: int16_t QueryInterface(T* me, const UGUID* iid, void out), uint32_t AddRef(T* me), uint32_t Release(T* me), где T — тип интерфейса. Успешный QueryInterface возвращает принадлежащую клиенту ссылку. Неподдерживаемый IID дает ECO_E_INTERFACE и *out=NULL. Последний Release освобождает все принадлежащие объекту ресурсы. Поддерживай канонический IEcoUnknown. Счетчики ссылок защищай от переполнения; совместное использование одного экземпляра требует внешней синхронизации. Ненулевой outer в фабрике отвергай явно.

6. Числовые типы и сервис. В include/IEcoDSP.h объяви EcoComplexF {float re, im;}, EcoComplexD {double re, im;} и EcoComplexL {long double re, im;}. Не используй complex.h. После базовых трех методов IEcoDSP расположи методы в следующем порядке:

```
int16_t fft(IEcoDSP*, uint32_t, const EcoComplexD*, EcoComplexD*);
int16_t ifft(IEcoDSP*, uint32_t, const EcoComplexD*, EcoComplexD*);
int16_t fftf(IEcoDSP*, uint32_t, const EcoComplexF*, EcoComplexF*);
int16_t ifftf(IEcoDSP*, uint32_t, const EcoComplexF*, EcoComplexF*);
int16_t fftl(IEcoDSP*, uint32_t, const EcoComplexL*, EcoComplexL*);
int16_t ifftl(IEcoDSP*, uint32_t, const EcoComplexL*, EcoComplexL*);
int16_t ffti(IEcoDSP*, uint32_t, const int*, EcoComplexD*);
int16_t iffti(IEcoDSP*, uint32_t, const int*, EcoComplexD*);
int16_t fftlong(IEcoDSP*, uint32_t, const long*, EcoComplexD*);
int16_t ifftlong(IEcoDSP*, uint32_t, const long*, EcoComplexD*);
int16_t CreateVector(IEcoDSP*, uint32_t, IEcoVector**);
int16_t CreateMatrix(IEcoDSP*, uint32_t, uint32_t, IEcoMatrix**);
int16_t CreatePlan(IEcoDSP*, uint32_t, IEcoFFTPlan**);
int16_t CreateFilter(IEcoDSP*, IEcoVector*, IEcoDigitalFilter**);
int16_t Filter(IEcoDSP*, IEcoVector*, IEcoVector*, IEcoVector*);
long double Add(IEcoDSP*, long double, long double);
long double Subtract(IEcoDSP*, long double, long double);
uint32_t Algorithm(IEcoDSP*);
uint32_t MemoryBytes(IEcoDSP*);
```

В таблице это указатели на методы с ECOCALLMETHOD, а не свободные функции. Первый uint32_t в fft-семействах задает N; затем идут вход и выход. В CreateMatrix аргументы задают rows и cols. В Filter порядок векторов: signal, kernel, output. Целочисленные входы интерпретируй как действительные значения; результат остается комплексным. Возвращай алгоритм 0 для DIT и 1 для DIF.

7. Интерфейсы данных и состояния. После трех базовых методов расположи следующие операции в указанном порядке. В IEcoVector: Size, Get, Set, MemoryBytes. Size и MemoryBytes возвращают uint32_t; Get принимает me, индекс uint32_t и EcoComplexL* out; Set принимает me, индекс и EcoComplexL по значению; оба возвращают int16_t. В IEcoMatrix: Rows, Cols, Get, Set, MemoryBytes; Get/Set дополнительно принимают два индекса row и col. В IEcoFFTPlan: Load(me, IEcoVector*), FFT(me), IFFT(me), Store(me, IEcoVector*), Size(me), MemoryBytes(me); первые четыре возвращают int16_t, последние два — uint32_t. В IEcoDigitalFilter: Apply(me, IEcoVector* signal, IEcoVector* result), Size(me), MemoryBytes(me); Apply возвращает int16_t, остальные — uint32_t. Параметр me в каждом случае имеет тип соответствующего интерфейса.

8. Математический контракт. Прямое преобразование вычисляет $X[k]=\Sigma x[n]\cdot\exp(-i\cdot 2\pi kn/N)$. Обратное вычисляет $x[n]=(1/N)\Sigma X[k]\cdot\exp(+i\cdot 2\pi kn/N)$. Сохраняй естественный порядок частот; не применяй fftshift. Принимай N от 1 до 65536 включительно. N=0 и превышение предела отвергай. Для N=1 возвращай исходный элемент. Точное совпадение входного и выходного массива поддерживай; частичное перекрытие не включай в контракт.

9. Два алгоритма. Реализуй отдельные итеративные radix-2 DIT и DIF. В DIT перестановка битов предшествует бабочкам, длины блоков возрастают, множитель применяется до сложения. В DIF длины убывают, множитель применяется к разности, битовая перестановка завершает расчет. Каждый алгоритм публикуй отдельной фабрикой, отдельной .a и отдельной .so. Имена библиотек определяются полным CID; динамический экспорт — GetIEcoComponentFactoryPtr. Для статической линковки предоставь GetEcoFFTDITFactory, GetEcoFFTDIFactory и символы фабрик с полным CID.

10. Произвольная длина. Для N, не равного степени двойки, реализуй алгоритм Блюстейна с M=nextpow2(2N−1). Вычисляй именно N-точечный спектр, а не FFT исходного сигнала, дополненного до другой длины. Используй chirp $\exp(-i\pi n^2/N)$, симметричное ядро свертки и внутренние FFT/IFFT выбранного DIT/DIF. Отдельно учитывай деление на M внутренней IFFT и на N внешней IFFT. До перехода к вещественному углу вычисляй n^2 modulo 2N безопасной целочисленной арифметикой. Обеспечь O(N log N) времени и O(N) памяти.

11. Численная реализация. Внутреннюю арифметику выполняй в long double. Поворотные множители вычисляй самостоятельно: приведение угла по четвертям к $[-\pi/4,\pi/4]$, ряды синуса и косинуса с 12 обновлениями после начального члена, восстановление четверти. Рассчитывай каждый коэффициент независимо; не накапливай фазу длинной цепочкой умножений. float и double расширяй перед расчетом и сужай только при выдаче результата. Конечные части входа ограничь величиной 10^{30} для float и 10^{100} для остальных типов. NaN/Inf отвергай без вызовов libc. Вектор, матрица и сохраняемое состояние плана используют предел 10^{100}. Опиши область применимости roundtrip при больших амплитудах.

12. Сервисный и объектный подходы. Сервис IEcoDSP хранит только инфраструктуру: vtable, счетчик ссылок, аллокатор и выбор алгоритма. Входной сигнал, таблицы и рабочие массивы сервисного вызова освобождаются до возврата. План владеет фиксированным N, таблицами, рабочими массивами и собственным сигналом. Load копирует данные; последующее изменение внешнего вектора не влияет на план. FFT/IFFT работают с внутренним сигналом без внешнего массива. Store копирует состояние в вектор клиента. До Load преобразования и Store возвращают ECO_E_STATE. После успешного создания повторные Load/FFT/IFFT/Store не выделяют память. Используй staging: ошибка чтения, проверки или преобразования сохраняет прежнее корректное состояние плана.

13. Фильтрация и контейнеры. Вектор и матрицу реализуй через собственные объекты; матрица имеет построчное хранение и не более 65536 элементов. Перед умножением размеров проверь переполнение. Фильтр выполняет N-точечную циклическую свертку $y[n]=\Sigma\ h[j]x[(n-j) \bmod N]$. При создании копируй ядро и сохраняй его спектр. Apply принимает новый сигнал, использует сохраненное ядро, не выделяет память и не меняет коэффициенты. Сервисный Filter создает временный фильтр на один вызов. Проверяй размеры, значения и совпадение входного/выходного вектора. Не приписывай этому API потоковую историю, IIR или двумерное FFT.

14. Ресурсы и ошибки. В компоненте используй только IEcoMemoryAllocator1 для выделения и освобождения памяти. Каждый объект удерживает собственную ссылку на аллокатор; откат частичного конструктора освобождает все уже созданные буферы. Выход конструктора при ошибке равен NULL. Коды: ECO_OK=0, ECO_E_ARGUMENT=−1, ECO_E_MEMORY=−2, ECO_E_RANGE=−3, ECO_E_STATE=−4, ECO_E_INTERFACE=−5. При ошибке аргумента или выделения памяти сервисный метод не записывает массив результата. Ошибку Get/Set стороннего IEcoVector передавай вызывающему; не обещай откат уже успешных записей внешнего контейнера. MemoryBytes сообщает запрошенные байты объекта и собственных буферов, без служебных данных аллокатора и памяти клиента.

15. Запрет стандартной библиотеки. В src/ и include/ не вызывай malloc, calloc, realloc, free, memcpy, memmove, memset, printf, assert, sin, cos, sqrt, pow и другие функции libc/libm; не включай системные заголовки. Копирование, заполнение, сравнение GUID и комплексную арифметику реализуй собственными циклами и операциями. Если depend.h SDK косвенно подключает libc, используй минимальный адаптер точных примитивных ABI-типов Linux x86_64, сохраняя оригинальные UGUID и vtable. Собирай ядро с -ffreestanding, -fno-builtin, -fno-tree-loop-distribute-patterns и без fast-math. .so линкуй с -nostdlib, -nodefaultlibs и -Wl,-z,defs. Проверь nm -u, readelf -d и реальные зависимости препроцессора. У обеих .so должны отсутствовать неопределенные символы и DT_NEEDED. В консольном клиенте и независимых эталонах стандартная библиотека разрешена.

16. Консольный интерфейс. Создай bin/eco_fft с операциями --fft и --ifft, обязательным --input=FILE, необязательным --output=FILE, параметрами --algorithm=dit|dif, --architecture=service|object и --dynamic=DIRECTORY. По умолчанию используй DIT, сервис

и статическую линковку. CSV: два столбца real,imag, одна строка на отсчет; необязательный заголовок, UTF-8 BOM, LF/CRLF, пробелы и строки комментариев #. Поддержи - для стандартных потоков. В режиме преобразования stdout содержит только CSV, ошибки идут в stderr, код завершения ненулевой при отказе. Сначала полностью прочитай и преобразуй вход, затем открывай выходной файл. Не перезаписывай результат при ошибке ввода. Реализуй --help, --test, --demo, --benchmark. Предоставь два CSV-примера и самостоятельное examples/use_component.c, получающее компонент через Eco runtime.

17. Функциональная проверка. Проверь оба компонента при статической и динамической загрузке. Независимо вычисляй прямое ДПФ в клиенте через sinl/cosl из math.h; не используй код проверяемого ядра в эталоне. Проверь FFT и IFFT для пяти типов, размеров 1,2,3,5,7,8,16,31,64,127,256,257 и пяти сигналов: импульс, константа, действительная гармоника, комплексная гармоника, фиксированная псевдослучайная последовательность. Перед эталоном квантуй вход в проверяемый тип. Для N=4096,65535,65536 проверь восстановление и Парсеваля. Добавь аналитический спектр [1,2,3,4], порядок частот, знак, нормировку, in-place, независимость планов, SOA/OOA и циклическую свертку. Пороговые значения ошибок обоснуй типом и амплитудой; не выдавай конечный эксперимент за доказанную точность всех входов.

18. Проверка отказов и клиента. Проверь NULL, неверные IID, нулевые/предельные размеры, индексы, несовместимые размеры, NaN/Inf, состояние до Load, ошибки внешнего Get/Set и владение ссылками. Через tracking allocator последовательно откажи на каждой точке выделения конструктора и сервисного вызова; после отказа не должно быть утечек. Сверь MemoryBytes с фактическими запросами. Запусти AddressSanitizer и UndefinedBehaviorSanitizer. Для CLI сравни результаты с независимым ДПФ на Python; проверь сервис/объект, DIT/DIF, FFT/IFFT, обе загрузки, стандартные потоки, пути с пробелами, BOM/CRLF, N=65536, N>65536, неверные CSV, сохранность существующего выхода при ошибке и сбои записи. Интеграционный пример должен собираться и реально выполняться.

19. Измерения. Сохрани прямой ДПФ на math.h для точности всех числовых типов; для сравнения времени со стандартным аналогом создай отдельный проверочный клиент с FFTW. Подключай FFTW только к этому клиенту, через необязательную CMake-опцию; компонент и основное приложение должны собираться без нее. Сравни DIT/DIF, FFT/IFFT и SOA/OOA на одинаковых данных long double, включая степени двойки, простые и составные длины до 65536. Нормируй FFTW_BACKWARD делением на N и проверь точность до замеров. Используй CLOCK_MONOTONIC, прогрев, подбор пакета длительностью не менее 10 мс с пределом 65536 повторов и медиану семи пакетов. Отдельно измеряй полный вызов с созданием/освобождением ресурсов и повторное применение готового плана с копированием данных; включай нормировку IFFT во время. Отдельно сохраняй создание/удаление планов, явно описав различие в выделяемых буферах. Фиксируй FFTW_ESTIMATE, один поток, прогретый внутрипроцессный кэш, CPU, компилятор, версии библиотек, параметры сборки и ограничения измерения. Публикуй CSV, команду воспроизведения и таблицы отчета; не выдавай самописный FFT

на math.h за стандартную FFT-библиотеку. Измерь память сервиса, вектора, плана и фильтра, отделив постоянные байты от пиковых временных. Никакие числа и успешные проверки не придумывай.

20. Поставка и документация. Оставь исходники, публичные интерфейсы, минимально необходимый SDK с атрибуцией и происхождением, Dockerfile, compose.yaml, CMakeLists.txt, скрипты сборки/проверок, примеры и готовые bin/ и lib/. Промежуточные файлы создавай в build/. Не включай кэши, исследовательские загрузки, копии каркасов, старые архивы и дубли документов. README начни с краткого назначения, затем приведи команды сборки, обработки собственного сигнала и подключения из C. Сохрани один документ docs/EcoFFT.docx: титульный лист с заполняемыми реквизитами, математика, архитектура, ABI/API, алгоритмы, память, точность, производительность, использование, ограничения, системный промпт и технические источники. Пиши документацию как описание самостоятельного программного продукта.

21. Условия завершения. Собери очищенную поставку с нуля в Docker. Проверь обе формы библиотек, реальные входные CSV, пример C, санитайзеры, сравнение с FFTW и отсутствие импортов ядра. Если проверка не проходит, найди причину, исправь код и повтори относящиеся к изменению проверки. Согласуй README и документ с фактическими именами файлов и командами. Заверши работу только после успешной проверки; перечисли созданные артефакты и фактические результаты. Невыполненные проверки обозначай явно, не подменяй их обещанием.

(конец системного промпта)

15.3. Почему требования сформулированы явно

Уточнение в промпте	**Какое нарушение предотвращает**	**Способ проверки**
Формулы знака, нормировки и порядка частот	Взаимно компенсирующиеся ошибки FFT/IFFT	Аналитический спектр и независимое ДПФ
Блюстейн для произвольного N	Изменение частотной сетки скрытым дополнением сигнала	Простые и составные нестепенные длины
Раздельные DIT/DIF и отдельные CID	Два названия одной реализации	Код бабочек и загрузка двух компонентов
Собственный сигнал плана и неизменяемое ядро фильтра	Формальное объектное API без состояния	Изменение внешнего вектора после Load/CreateFilter
Аудит ELF и зависимостей препроцессора	Неявный вызов libc из SDK или оптимизации компилятора	nm, readelf, -M и freestanding-сборка
Отказы на каждой точке выделения	Утечки частичных конструкторов	Tracking allocator и ASan/UBSan
Отмена регистрации до выгрузки .so	Обращение шины к уже выгруженной таблице методов	Динамические тесты и порядок Release/UnRegister
CSV до открытия результата	Потеря существующего файла при неверном входе	Проверка сохранности содержимого

Уточнения связывают требование с наблюдаемым поведением и конкретным способом проверки. При развитии API промпт, заголовок и приемочные сценарии должны обновляться согласованно; существующие IID и порядок vtable нельзя менять без отдельного решения о совместимости.

16. Ограничения и направления развития

EcoFFT предоставляет два ACOM-компонента для FFT/IFFT, единый сервисный контракт и объекты с собственным состоянием. Векторы, матрицы, планы и фильтры создаются через официальный Eco-аллокатор. DIT/DIF имеют отдельные реализации и идентификаторы; произвольные длины обслуживаются через Блюстейна. Компонент доступен как из C-приложения, так и через консольный CSV-интерфейс.

Сервис удобен для разового вычисления и требует меньше постоянной памяти. План сохраняет коэффициенты для повторных расчетов одного размера, но увеличивает постоянную память и затраты на Get/Set и копирование. Измерения показывают фактический баланс этих факторов; наличие кэша не обещает ускорения для каждого размера. Блюстейн требует дополнительных буферов и преобразований по сравнению с radix-2.

Текущая платформа — Linux x86_64; $N \le 65536$. Каждый объект используется одним потоком либо под внешней синхронизацией. Матрица хранит данные, фильтр выполняет циклическую блочную свертку. Измеренная точность относится к проверенным сигналам и форматам. SIMD, многопоточность, двумерное преобразование, потоковая фильтрация и автоматический выбор плана остаются направлениями дальнейшей разработки. Сравнение с FFTW в режиме FFTW_ESTIMATE приведено в разделе 12; другие режимы планирования и MATLAB не измерялись.

17. Технические источники

[1] Eco.AI Agentic Coding Meta-Harness. Официальный репозиторий: https://github.com/peerf-eco/eco-harness-releases .

[2] Релиз v0.1.3, SDK и инструменты: https://github.com/peerf-eco/eco-harness-releases/releases/tag/v0.1.3 . Происхождение и SHA256 сохраненных SDK-файлов зафиксированы в vendor/eco/provenance.json.

[3] Cooley J. W., Tukey J. W. An Algorithm for the Machine Calculation of Complex Fourier Series. Mathematics of Computation, 1965, 19(90), 297–301. DOI: https://doi.org/10.1090/S0025-5718-1965-0178586-1 .

[4] Smith J. O. Mathematics of the DFT. Bluestein’s FFT Algorithm. https://www.dsprelated.com/freebooks/mdft/Bluestein_s_FFT_Algorithm.html . Использован математический принцип сведения к свертке; программная реализация самостоятельная.

[5] FFTW. What FFTW Really Computes / Complex One-Dimensional DFTs. https://www.fftw.org/doc/What-FFTW-Really-Computes.html ; https://www.fftw.org/doc/Complex-One_002dDimensional-DFTs.html . Определение преобразований, знак, нормировка и C API библиотеки сравнения; вычислительный компонент от FFTW не зависит.

[6] FFTW. Precision. https://www.fftw.org/doc/Precision.html . Интерфейс fftwl_* и библиотека libfftw3l для long double.

[7] FFTW. Planner Flags. https://www.fftw.org/doc/Planner-Flags.html . Режим FFTW_ESTIMATE, повторное использование wisdom и ограничения планирования.